SparseGate: Mixture-of-Experts Routing with Learned Sparsity

Priya Nair, Tomas Berg and Hannah Cole. Fictional paper written for testing PaperMind.

Abstract

We introduce SparseGate, a mixture-of-experts routing method that learns how many experts each token should use. SparseGate uses 8 experts per layer with top-2 routing and an auxiliary load-balancing loss. On the NLU-Bench suite it reaches an average accuracy of 86.1, outperforming a dense 7-billion-parameter baseline at 85.0, while activating only 1.6 billion of the 7 billion parameters for each token. Training costs 412 GPU-hours.

1 Introduction

Dense language models spend the same compute on every token, whether the token is a common function word or a rare technical term. Mixture-of-experts (MoE) models address this by routing each token to a small subset of expert networks, increasing capacity without increasing per-token compute.

Existing routers have two weaknesses. First, they often suffer routing collapse, where a few experts receive most tokens and the others are never trained. Second, they use a fixed number of experts per token. SparseGate addresses both with a learned sparsity penalty and a load-balancing objective. Our contributions are a noisy top-k router with a learned sparsity term, a load-balancing loss that prevents collapse, and state-of-the-art results on NLU-Bench at lower inference cost than a dense model.

2 Related Work

Sparsely-gated mixture-of-experts layers were proposed to scale model capacity cheaply. Later work simplified routing to a single expert per token. Load-balancing losses are widely used but are usually tuned by hand. Our approach differs by learning sparsity jointly with routing and by reporting how often training collapses without the balancing term.

3 Method

Each transformer layer replaces its feed-forward block with 8 experts. A linear router scores every expert for every token. During training we add Gumbel noise to the scores and keep the top 2 experts, renormalizing their weights with a softmax. A sparsity penalty with coefficient 0.01 encourages the router to put most of its probability mass on a single expert when the token is easy.

To prevent routing collapse we add an auxiliary load-balancing loss with coefficient 0.02 that penalizes uneven expert usage within each batch. The full model has 7 billion total parameters, of which about 1.6 billion are active for any single token.

4 Experiments

We train on the same pretraining mixture as the dense baseline and evaluate on NLU-Bench, the same nine-task suite used in prior parameter-efficient work. We compare against a dense 7-billion-

parameter model that is fully fine-tuned. Training uses 16 A100 GPUs for a total of 412 GPU-hours, and every result is the mean of three seeds.

5 Results

SparseGate reaches an average NLU-Bench accuracy of 86.1, compared with 85.0 for the dense baseline, a gain of 1.1 points. Because only 1.6 billion parameters are active per token, inference throughput is 1.9 times higher than the dense model at equal batch size.

Removing the load-balancing loss causes routing collapse in 5% of training runs and lowers accuracy to 83.4 in the runs that do converge. Using top-1 routing instead of top-2 reduces accuracy to 84.6, while top-4 routing adds cost without improving accuracy.

6 Limitations

Although few parameters are active per token, all 7 billion parameters must be kept in memory, so SparseGate does not reduce memory requirements. Multi-GPU deployments incur communication overhead when tokens are sent to experts on different devices. Training also costs 412 GPU-hours, which is far more than parameter-efficient fine-tuning methods that train only a small adapter.

7 Conclusion

SparseGate demonstrates that learned sparsity and load balancing make mixture-of-experts routing both stable and accurate. It beats a dense baseline on NLU-Bench while using a fraction of the compute per token. Future work will study expert placement and reducing the memory footprint.

References

[1] D. Author. Sparsely-gated mixture-of-experts layers. 2017. [2] E. Writer. Switch transformers. 2021. [3] C. Scholar. The NLU-Bench evaluation suite. 2023.